

Experiment 9: A Search Algorithm*

Aim

To implement the A* Search algorithm in Python.

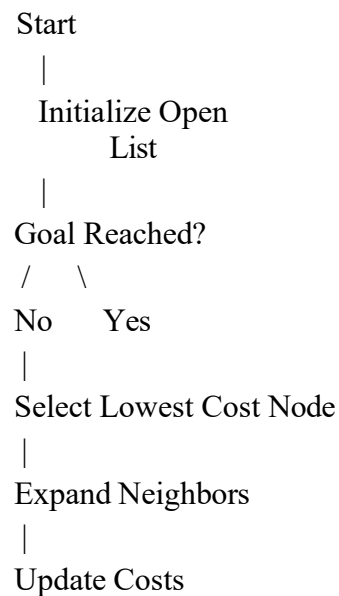
Objective

- To find the shortest path between two nodes.
- To use heuristic values for efficient searching.

Algorithm

1. Initialize open and closed lists.
2. Add the start node to the open list.
3. Select the node with minimum $f(n)=g(n)+h(n)$.
4. Expand neighboring nodes.
5. Update costs if a better path is found.
6. Repeat until the goal node is reached.

Flowchart



```
|  
Repeat  
|  
End
```

Program

```
graph = {  
  
    'A': [('B',1),('C',3)],  
  
    'B': [('D',3),('E',1)],  
  
    'C': [('F',5)],  
  
    'D': [],  
  
    'E': [('F',2)],  
  
    'F': []  
  
}  
  
heuristic = { 'A':5,  
  
    'B':4,  
  
    'C':3,  
  
    'D':2,  
  
    'E':1,  
  
    'F':0  
  
}  
  
open_list=[('A',0)]  
  
visited=[]
```

```
while open_list:

    open_list.sort(key=lambda x:x[1]+heuristic[x[0]])

    node,cost=open_list.pop(0)

    if node in visited:

        continue

    print(node,end=" ")

    if node=='F':

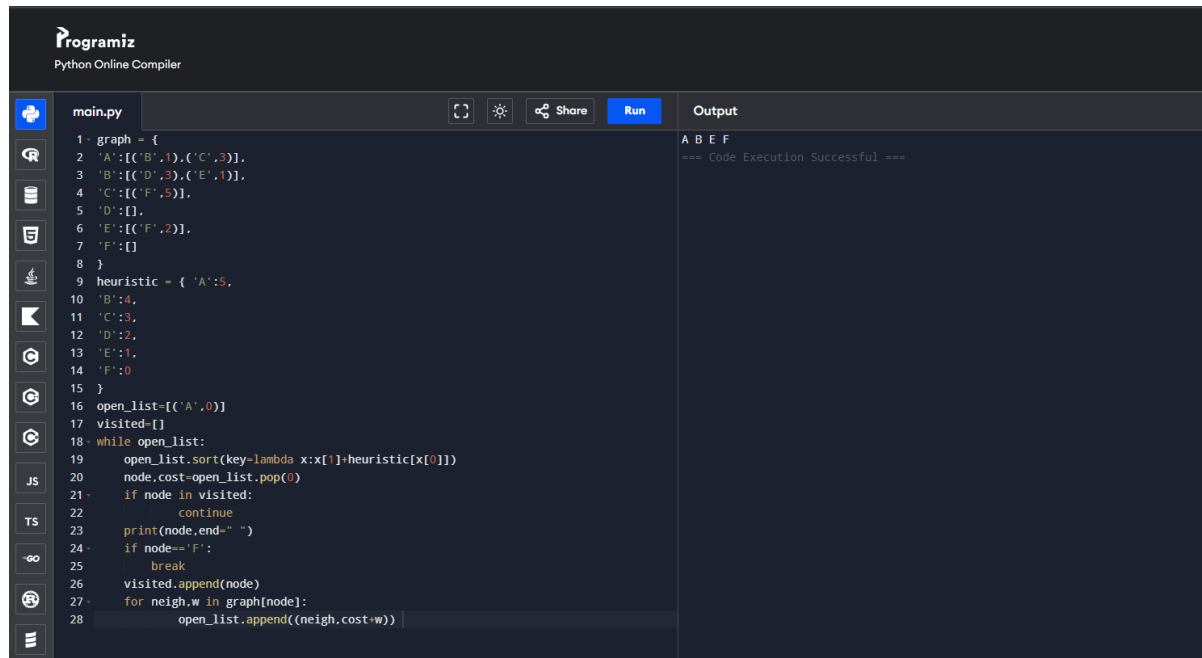
        break

    visited.append(node)

    for neigh,w in graph[node]:

        open_list.append((neigh,cost+w))
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The left sidebar contains icons for various programming languages and a file explorer. The main editor displays a Python script for the A* algorithm. The script defines a graph with nodes A through F, their neighbors, and heuristic values. It then implements the A* search algorithm, starting from node A and finding the shortest path to node F. The output panel on the right shows the path 'A B E F' and a success message.

```
1 graph = {
2   'A':[(('B',1),('C',3))],
3   'B':[(('D',3),('E',1))],
4   'C':[(('F',5))],
5   'D':[],
6   'E':[(('F',2))],
7   'F':[]
8 }
9 heuristic = { 'A':5,
10  'B':4,
11  'C':3,
12  'D':2,
13  'E':1,
14  'F':0
15 }
16 open_list=[('A',0)]
17 visited=[]
18 while open_list:
19     open_list.sort(key=lambda x:x[1]+heuristic[x[0]])
20     node,cost=open_list.pop(0)
21     if node in visited:
22         continue
23     print(node,end=" ")
24     if node=='F':
25         break
26     visited.append(node)
27     for neigh,w in graph[node]:
28         open_list.append((neigh,cost+w))
```

Output

```
A B E F
=== Code Execution Successful ===
```

Result

The shortest path was found successfully using the A* algorithm.

Conclusion

A* efficiently finds the shortest path using both path cost and heuristic information.